Molecular Dynamics Onboarding

These are notes on learning molecular dynamics (MD) for the first time, for people with at least some experience coding and a good sense of vector calculus. I'm writing for the first time in 2024, any comments are welcome.

1 Energy Conservation with Velocity Verlet

In most of the simulations we work with things have harmonic potentials, mainly this is to make our lives simpler, we treat pretty much everything as a linear force law approximation so we end up with dealing with a lot of springs and 'spring like' things[1].

First set of tasks: For each system described below, (1) derive the forces on the particles by taking the gradient of the potential. (2) Plot a representation of the system so you can visualize what is going on. (3) Then implement an energy conserving simulation using the Verlet integration (I like the version described right beneath "Eliminating the half-step velocity" on the wikipedia page). (4) check that the energy of the system is conserved and that over the course of a simulation where energy is transferred between kinetic and potential frequently the standard deviation of the total energy E, is propotional to the time step squared:

$$\text{std}(E) \propto \Delta t^2 \tag{1}$$

. Simulations will consist of one or more particles moving around and possibly bumping into the walls of a box containing them or one another.

1.1 Verlet Integration

A Verlet step updates forces, F, velocities, v, and positions, x, with $\text{O}(\Delta t^2)$ error in the conserved energy[2]. A step from t to $t + \Delta t$ can be written:

$$x(t + \Delta t) = x(t) + v(t)\Delta t + \frac{1}{2m}F(x(t))\Delta t^2 \tag{2}$$

$$v(t + \Delta t) = v(t) + \frac{\Delta t}{2m}(F(x(t)) + F(x(t + \Delta t))) \tag{3}$$

Or in code:

[1] 'Spring like' can be usefully because they are simple, but also because as you approach a static system we can approximate all potentials to second order with 'spring like' harmonic potentials using taylor expansions.

[2] When $\Delta t \ll 1$ we can get far more accurate simulations from an integration scheme with errors of order Δt^2 than something like Euler integration which would be something like $x(t + \Delta t) = x(t) + \Delta t v(t)$ and $v(t + \Delta t) = v(t) + \Delta t F(t)/M$.

Initialize x, v, F;
while *As Long as you want to run it* **do**
  $x(t + \Delta t) = x(t) + \Delta t * v(t) + \frac{\Delta t^2}{2*m} * F(t)$;
  Compute $F(t + \Delta t)$; `/* Usually` $-\nabla U(x(t + \Delta t))$ `*/`
  $v(t + \Delta t) = v(t) + \frac{\Delta t}{2*m} * (F(t + \Delta t) + F(t))$
  Compute Energy $E(t + \Delta t)$; `/* Or any other state variables. NB order matters in these integration schemes. For example if you calculate the kinetic energy before updating the velocities you will end up with error in energy that goes as` Δt^1 `*/`
end

Algorithm 1: Velocity Verlet Psuedocode

1.2 Systems:

1. 1D Spring. A particle with mass m is connected to a 1D spring at point x_0. The potential energy is:

$$U = \frac{k}{2}(x - x_0)^2 \tag{4}$$

2. Attracted particles. N 'soft disks' in 2D (soft disks just means disks that repel with a spring like interaction) are in a bowl like well. They have diameter σ and interact like purely repulsive springs. They also interact with a potential energy that pulls them towards the center the system with a gravity force. The total potential energy is:

$$U = U_{\text{int}} + U_{\text{center}} \tag{5}$$

The interaction potential is:

$$U_{\text{int}} = \sum_{i=1}^{N-1} \sum_{j=i+1}^{N} \frac{k_{\text{int}}}{2} \left(\sigma - r_{ij}\right)^2 \Theta(\sigma - r_{ij}) \tag{6}$$

where $r_{ij} = |\vec{r}_j - \vec{r}_i|$, k_{int} is the spring constant for these interactions, $\vec{r}_i$ is the center of the ith disk, $\Theta(\cdot)$ is the Heaviside step function, it is 1 when its argument is greater than zero and zero otherwise. The central attraction potential is:

$$U_{\text{center}} = \sum_{i=1}^{N} \frac{k_{\text{center}}}{2} \left(|\vec{r}_i - \vec{r}_0|\right)^2 \tag{7}$$

where k_{center} is the strength of the spring pulling everything towards the center and $\vec{r}_0$ is the center of the system.[3]

[3] If it helps with intuition: you can imagine the particles in the central potential to be a

3. Particles in a box. In the above example replace the U_{center} term with U_{walls} The wall potentials is:

$$\begin{aligned} U_{\text{walls}} = \sum_{i=1}^{N} \frac{k_{\text{walls}}}{2} \Big[& (\sigma - x_i)^2 \, \Theta(\sigma - x_i) + \\ & (x_i - (L_x - \sigma))^2 \, \Theta(x_i - (L_x - \sigma)) + \\ & (\sigma - y_i)^2 \, \Theta(\sigma - x_i) + \\ & (y_i - (L_y - \sigma))^2 \, \Theta(y_i - (L_y - \sigma)) \Big] \end{aligned} \tag{8}$$

where L_x and L_y are the x and y length of the box.

4. Particles in a periodic box. A commonly used tool in MD is putting your system in a periodic box to isolate the system from boundary effects (really you're just introducing a set of different boundary effects, but it is still useful). In a periodic box disks will only interact with one another, so the only potential energy is U_{int} from above. When we say we are using a periodic box, we take that to mean that the distance between two particles is the shortest distance between particle i and any of the copies of particle j on the square periodic grid, put into math what this means is:

$$r_{ij}^{\text{Periodic}} = (\vec{r}_j - \vec{r}_i) - \begin{bmatrix} L_x \text{round}\left(\frac{\vec{r}_{j,x} - \vec{r}_{i,x}}{L_x}\right) \\ \\ L_y \text{round}\left(\frac{\vec{r}_{j,y} - \vec{r}_{i,y}}{L_y}\right) \end{bmatrix} \tag{9}$$

we sometimes call r_{ij}^{periodic}, r'_{ij}. So here the potential energy is

$$U = U_{\text{int}} = \sum_{i=1}^{N-1} \sum_{j=i+1}^{N} \frac{k_{\text{int}}}{2} \left(\sigma - r'_{ij}\right)^2 \Theta(\sigma - r'_{ij}) \tag{10}$$

See Fig. 1 shows some example plots that you could try to recreate for each system. The figure shown is for particles in a non-periodic box.

bunch of marbles in a bowl (the height of which goes as r^2). In this equivalence the height of each particle $h_i = (|\vec{r}_i - \vec{r}_0|)^2$, and $k_{center}/2$ would be like the mass times the gravity constant, $m_i g$. So in these terms the potential energy of the ith particle would be $m_i g h_i$. For the purposes of this excersize it is easier to do things in 2D, but this might help motivate why we would want a central attractor like this.

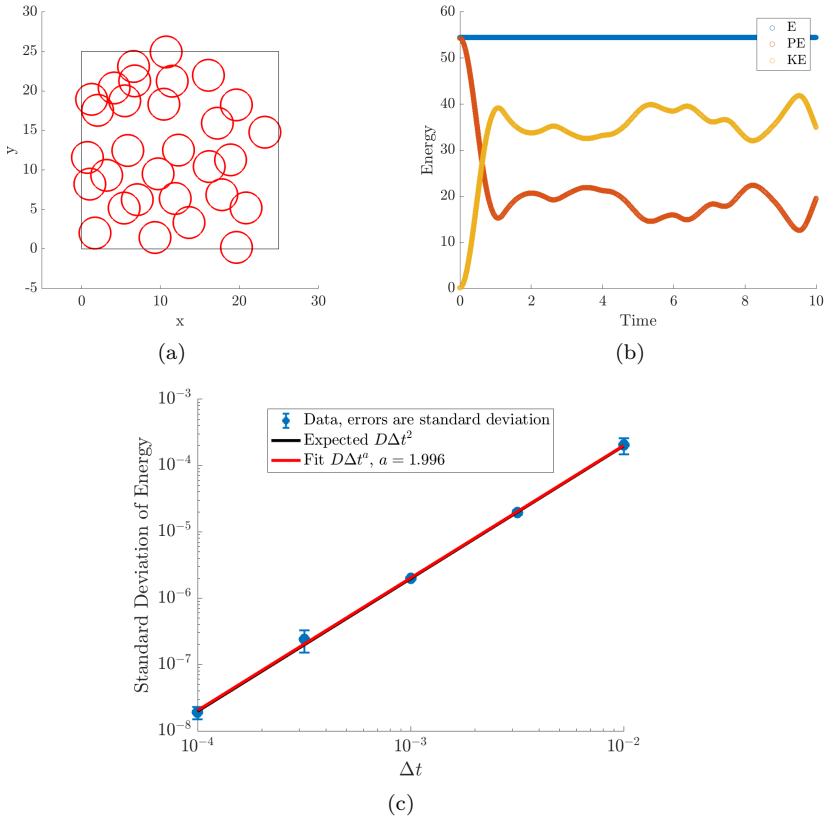

Figure 1: (a) Visualization of the system. (b) Potential and kinetic energy overtime, note there is a 'burn in' period at the start because I start the system with zero velocity and non-zero overlap, then some of the potential energy gets turned into kinetic energy. (c) Power law for $\text{std}(E) \propto \Delta t^2$. I calculate $\text{std}(E)$ using only time after the 'burn in' period. I vary Δt over several decades, but make sure the total amount of simulation time is the same, i.e. if you divide Δt by 10 you need to run for ten times as long. I also average over several different initial configurations for each data point.

1.3 Tips debugging and otherwise

1. Initializing. For a simulation to be useful it must be dynamic, to get dynamics, you need to either start a system with initial velocities, or have non-zero potential energies (i.e. overlaps) when you start. For the mass on the spring you can initialize it with $x(t = 0) \neq x_0$, for the disks in the central attractor, as long as they are not at $\vec{r}_0$ they will move. For the packings of particles in boxes you can either start the particles with some initial velocity (in different directions), or with some overlaps. If

you are initializing the positions randomly within the box you can use the following as a rule of thumb: the packing fraction of the system is the amount of space particles take up over the the total space, here,

$$\phi = \frac{N\pi(\sigma/2)^2}{L_x L_y}, \tag{11}$$

when $\phi < 0.5$ it is possible to have no overlaps at initialization, when $\phi > 0.8$ you are guaranteed to have some overlaps for disks with all the same diameter.

2. Picking parameters. To make life simple you can pick most things to be one. I suggest using $\sigma = 1$, all $k = 1$, for picking the time step it is best to have it be much smaller than the fastest harmonic. To find the periodic of the fastest harmonic you take the largest spring constant, $k_{\max}$ and the smallest mass, $m_{\min}$

$$P = 2\pi\sqrt{\frac{m_{\min}}{k_{\max}}} \tag{12}$$

and we set:

$$\Delta t = \frac{P}{100} \tag{13}$$

. This should be the largest time scale you at which you should expect stable results.

3. If you have bugs simplify the system as much as you can and try to find the line of code where the bug first appears. Start in 1D, then go to 2D, and possibly 3D.

4. You can calculate forces by hand based on the positions and compare those to your calculated forces

5. Look at what you're doing, plot the system over time and over the few time steps where an unexpected behavior begins, try plotting forces and velocities too, are they what you expect? To plot disks (circles) of a particular radius the `rectangle` function in matlab is useful. Search for "Draw Circle" on the matlab docs page.

6. Also try plotting the energy kinetic, potential, versus time or even break it up into smaller terms to find where things are going wrong.

7. Reach out! Let me know how you're doing – I'm happy to answer questions or clarify anything!

8. A full simulation code might look something like this:

```
x = InitPositions(N) ; /* Initialize x, v, F for N particles */
v = 0 * x;
F = 0 * x ;
E(1:T) = 0;
K(1:T) = 0;
U(1:T) = 0 ;                                  /* Main Simulation loop */
for ( t = 1, t < N_steps, t = t + 1 ) {
    F_previous = F;
    x = x + Δt * v + Δt^2/(2*m) * F;
    [F, U(t)] =-gradU(x);
    v = v + Δt/(2*m) * (F + F_previous);
    K(t) = 1/2 Σ m v_i^2;
    E(t) = U(t) + K(t) ; /* Save anything you want to plot here
     */
}
; /* Plotting code                                                  */
plot(1:T,U(1:T);
plot(1:T,K(1:T);
plot(1:T,E(1:T);
etc;
;
; /* Function Definitions                                          */
Function gradU(x):
    ...
    return F, U;
Function InitPositions(N):
    ...
    return x;
etc;
```

Algorithm 2: Simulation Psuedocode

1.4 Minimization Schemes

1. Damped MD (done below) (could be implemented through Langevin dynamics swell)
2. Overdamped MD
3. F.I.R.E.
4. Conjugate Gradient (to do)

2 Minimization Schemes

2.1 Damped Dynamics

Damped dynamics are very similar to Velocity-Verlet dynamics. The only change is that you have to add a damping term to the forces. Sometimes it can be useful to think about the force contributions from damping as separate from the rest of the forces (e.g. when looking at modes in the system). For the purposes of running damped dynamics I find it helpful to think of all the forces as one vector.

A damped Verlet step updates forces, F, velocities, v, and positions, x. A step from t to $t + \Delta t$ can be written.

$$x(t + \Delta t) = x(t) + v(t)\Delta t + \frac{1}{2m}F(t)\Delta t^2 \tag{14}$$

$$F(t + \Delta t) = F(x(t + \Delta t), v(t)) = -\nabla U(x(t + \Delta t)) - \beta v(t) \tag{15}$$

$$v(t + \Delta t) = v(t) + \frac{\Delta t}{2m}\left[F(t) + F(t + \Delta t)\right] \tag{16}$$

I have not been careful here about where I use $v(t)$ vs $v(t + \Delta t)$ to calculate damping forces so it likely is only accurate to first order in Δt. This wont matter if you are just trying to energy minimize to *some* minimum. However, if you are trying to simulate some kind of physical damping you should consider finding another source on this.

The kinetic energy in the system will decay exponentially, and eventually the system will approach an equilibrium state where the kinetic energy approaches zero. Note you must pick an energy cutoff to call 0 energy. You must also choose a damping coefficient β which has units of mass over time in this formulation. (Try $\beta = 1$ to start and play around with it.)

Or in code:

1 Initialize x, v, F;
2 **while** *As Long as you want to run it* **do**
3 $x(t + \Delta t) = x(t) + \Delta t * v(t) + \frac{\Delta t^2}{2*m} * F(t)$;
4 Compute $F(t + \Delta t)$; /* $-\nabla U(x(t + \Delta t)) - \beta v(t)$ */
5 $v(t + \Delta t) = v(t) + \frac{\Delta t}{2*m} * (F(t + \Delta t) + F(t))$
6 Compute Energy $E(t + \Delta t)$; /* Or any other state variables. NB order matters in these integration schemes. */
7 **end**

Algorithm 3: Damped Velocity Verlet Psuedocode

2.1.1 Overdamped dynamics

Another variation on damped dynamics are dynamics where the damping is so large that particles effectivly come to a stop at each time step so

$$\frac{\partial x}{\partial t} = \frac{1}{b}F(t) \tag{17}$$

$$\tag{18}$$

where b is related to β in principle but in practice is just choosing a time step.

You could write the position and velocity update as:

$$x(t + \Delta t) = x(t) + \Delta t \cdot v(t) \tag{19}$$

$$v(t + \Delta t) = \frac{\Delta t}{m}\frac{F(t)}{b} \tag{20}$$

If all particles have the same mass b and β can be thought of as a single variable. Again if you're going to use this in something more than energy minimization in a learning context please consult a more authoritative document, but this is the gist.

2.2 Fire Minimization of Energy

The original fire paper: 10.1103/PhysRevLett.97.170201, The modification with back stepping that I use arXiv:1908.02038v1. the algorithm 2 I have copied out below in Algorithm 6. It will yield a packing with a force balance given by the stopping criteria. Please give them a read (or at least a skim before proceeding).

A note on convergence. Many criteria are possible. A few things to bear in mind: convergence speed and achieveable criteria usually depend on the number of degrees of freedom in the system. One way to address this is to have a target that depends on this too, for example stopping criteria that checkes if the total potential energy at step i is below a threshold and (possibly or) has not changed much from the last step checked $i - 1$:

$$\frac{U_i}{N_{dof}} < S \cdot k_c\sigma^2 \tag{21}$$

and

$$\frac{1}{N_{dof}}\left(\frac{U_i}{U_{i-1}} - 1\right) < S \cdot k_c\sigma^2 \tag{22}$$

Where S is some small factor e.g. 10^{-12} Another way to do stopping criterai is to measure force balance, that is the maximum net force on a particle divided by the average force component. This can be fairly memory intensive because

you need to keep track of each contributing component. The criteria is:

$$\frac{\max_{i\in\text{vertices}} |F_i^{\text{net}}|}{\langle |F_j^{\text{component}}| \rangle_{j\in\text{contributions to } F_i^{\text{net}}}} < S \cdot N_{dof} \tag{23}$$

This way of thinking about it most intuitive for disks where the contacts are the only contributions

```
Preset N_max, Δt_start, α_start, N_delay, InitialDelay, f_inc ;
Δt_max, f_α, N_P≤0,max, f_dec, Δt_min, Convergence Criteria ;
;
Initialize x(t) and F(x(t));
v(t) = 0 ;
α = α_start ;
Δt = Δt_start ;
N_P>0 = 0;
N_P≤0 = 0;
for ( i = 1; i < N_max ) {
    P(t) = F(x(t)) · v(t) ;
    if P(t) > 0 then
        N_P>0 = N_P>0 + 1 ;
        N_P≤0 = 0 ;
        if N_P>0 > N_delay then
            Δt = min(δt * f_inc, δt_max) ;
            α = α * f_α ;
        end
    else
        N_P>0 = 0 ;
        N_P≤0 = N_P≤0 + 1 ;
        if N_P≤0 > N_P≤0,max then
            break;
        end
        if (i >= N_delay) and not InitialDelay then
            if Δt * f_dec >= Δt_min then
                Δt = Δt * f_dec ;
            end
            α = α_start ;
        end
        x(t) = x(t) − 0.5 * Δt * v(t) ;    /* Correct uphill motion */
        v(t) = 0 ;
    end
    Calculate x(t + Δt), v(t + Δt), F(x(t + Δt)), E(x(t + Δt)) ;    /* MD
     integration, in my case velocity Verlet mixing (with
     V_norm and F_norm see Algorithm 5 */
    t = t + Δt ;
    if Converged then
        break ;      /* I use a threshold on the maximum allowed
         force ideally around 10^−12 */
    end
}
```

Algorithm 4: Fire 2.0 Algorithm

1 $v(t + \frac{1}{2}\Delta t) = v(t) + \frac{1}{2}\Delta t * F(x(t))/m$;
2 MixingFactor $= |v(t + \frac{1}{2}\Delta t)|/|F(t)|$;
3 $v(t + \frac{1}{2}\Delta t) = (1 - \alpha) * v(t + \frac{1}{2}\Delta t) + \alpha * \text{MixingFactor} * F(x(t))$;
4 $x(t + \Delta t) = x(t) + \Delta t * v(t + \frac{1}{2}\Delta t)$;
5 Calculate $U(t + \Delta t)$;
6 $F(t + \Delta t) = -\nabla U(x(t + \Delta t)$;

Algorithm 5: Velocity Verlet Mixing

3 Creating Jammed Packings

3.1 Binary Search for Disks

Binary search for jammed packings. The packing must begin at a low density with pressure less than target pressure P_t (alternately a target potential energy U_t can be used). A range of acceptable pressures is set from the high value of pressure $P_{t,h}$ to the low value of pressure $P_{t,l}$. Usually they are one percent away from each other. A binary search like algorithm is used to find the a box size L that satisfies the target pressure.

At first $P < P_{t,l}$, and in each iteration L is shrunk by a fixed amount r_{scale}. Each time the algorithm finds an unjammed state it saves this state.

The loop: If the pressure is less than the lower bound of the target pressure, the state is saved, L_h is noted. If, currently a jammed state is known L is set to be in the middle of the known over-pressure and under-pressure states. Since we are at lower than threshold pressure – rearrangements are a concern and so L_l is unset as this may no longer correspond to a jammed packing. If L_l was no known at the beginning of this iteration, L is instead rescalled by a fixed amount r_{scale}.

If $P > P_{t,l}$ this is an over-pressure state. First the state is returned to the most recent under-pressure state. Then L_l is set to be the current L, and the new L is set to be the mean of the new L_l and the previously known L_h.

If $P_{t,l} < P < P_{t,h}$ its a success and the loop breaks. The loop also breaks if the lower and upper bounds are too close to one another and it looks impossible to find a target pressure state.

When there is a known over-pressure state (L_l is set) and we keep finding over-pressure states this works as a binary search algorithm. However due to the possibility of rearrangements we can't use the binary search method when we have an under-pressure state, hence the unsetting of L_l when we find under pressure states. In pseudo code the algorithm is as follows:

```
Set P_{t,l} ;                                                          /* lower pressure bound for the jammed state */
Set P_{t,h} ;                                  /* upper pressure bound for the jammed state, often P_{t,h} = 1.01 * P_{t,l}  */
Set E_thresh, N_minimization ;   /* energy minimization sensitivity - could replace with force as appropriate, and
  max number of minimization steps */
success = 0;
r_scale = 0.999 ;                                                                              /* rate of shrinking */
L_l = −1; L_h = −1;                                                          /* Lower and upper bound of box size */
Set L_tol ;                                                    /* Smallest space searched usually 1e-10 */
Initialize System, state.Positions, state.Velocities, box size L ;        /* Initialize state with positions and
  velocities */
Save, OldState = state;
while true do
    state = EnergyMinimize(state, L, E_thresh, N_minimization);
    L_prior = L ;                                                        /* save prior L for rescalling purposes.  */
    P = Pressure(state, L);
    if P < P_{t,l} then
        OldState = state ;                                                          /* save known unjammed state */
        L_old = L;
        L_h = L ;                                  /* when L_h > 0 it is the box length for a known unjammed state */
        if L_l > 0 then
            L = (L_h + L_l)/2 ;  /* split the difference between known jammed state and known unjammed state */
            L_l = −1 ; /* If P < P_{t,l} rearrangment possible, L_l may no longer correspond to unjammed state */
        else
            L = L · r_scale ;                  /* shrink the box by a fixed ammount if no jammed state is known */
        end
    else
        if P > P_{t,h} then
            L_l = L ;                                /* when L_l > 0 is is the box length for a known jammed state */
            state = OldState ;                                  /* reset to prior state and shrink a smaller amount */
            L_prior = L_old;
            L = (L_h + L_l)/2 ;  /* split the difference between known jammed state and known unjammed state */
        else
            success = true;
            break;
        end
    end
    state.Positions = state.Positions ·L/L_prior ; /* Rescale system to avoid boundary effects.  NB: be aware of
      added presress esp.  with many body potentials - this is optional and might be 'less physical' depending on
      what you're model of jamming is */
    if L_l > 0 and L_h > 0 and |L_h/L_l − 1| < L_tol then
        error("No Jammed State Found");                          /* no jammed state at this pressure could be found */
    end
end
```

Algorithm 6: Jamming Threshold